Small Juries, Early Verdicts

A calibrated Gemma 4 E4B jury reads the first ten steps of a SWE-agent run, predicts whether it will fail, and lets a fixed compute budget resolve 29% more issues

Francisco Angulo de Lafuente (Agnuxo1)

Gemma 4 Developer Agent Paper Track, November 2026

Abstract

Offline coding agents on a single accelerator work under a hard budget, and most of that budget is spent on attempts that end up failing. We ask whether a *small, local* judge can spot those attempts early. A Gemma 4 E4B "jury" reads only the issue and the first k agent steps of a SWE-agent run and scores six rubric dimensions (localization, reproduction, coherence, test evidence, loops, scope); a logistic layer fitted on development data turns them into a calibrated P(success). On 600 held-out runs from 453 repositories never used in development (pre-registered hypotheses, repository- cluster bootstrap, Holm correction), the jury predicts final success from the first 10 steps with AUROC 0.677 [0.619, 0.734] versus 0.575 for trace features without an LLM (Δ +0.102, Holm-adjusted p = 0.02); the pre-registered test at 20 steps was inconclusive (38 positives). In a budget simulation, aborting runs whose predicted failure exceeds a threshold chosen on development data resolves **+29% more tasks per agent step** [+14, +45] (Holm p ≤ 0.004), +19 points more than the same policy driven by trace features; it matches the best step cap tuned on development data while deciding ten steps earlier, and combined with that cap it reaches **+36%** [+19, +54]. The 2B-effective E2B keeps 93% of the AUROC at 55% of the GPU time. A Qwen3-4B control with identical prompts shows the effect is not Gemma-specific, and Gemma 4 E4B gives the strongest jury. We release code, prompts, all verdicts and a blind validation of the rubric.

1. Introduction

The Gemma 4 Developer Agent setting is austere: small open models, a fixed time budget, no internet. Every step spent on a doomed attempt is a step not spent on a promising one, and doomed attempts dominate: 83% of the 80,036 public SWE-agent runs we study are unresolved, and unresolved runs are longer. Running the tests answers "did it work?" only at the end, and needs a working environment per repository.

We study the earlier and cheaper question: **after a handful of steps, is this run worth continuing?** Our contributions:

1. **An early-verdict task and protocol.** Failure prediction from *absolute* step prefixes (k = 10, 20) of real SWE-agent trajectories, with repository-disjoint splits and a no-LLM trace-feature control. We show that fractional prefixes ("the first 66% of the run"), the natural first design, leak the final run length and make trace features look strong; absolute prefixes remove that leak.
2. **A small, interpretable jury.** Six rubric dimensions scored by Gemma 4 E2B/E4B and aggregated by a 7-parameter logistic layer fitted on development data. It discriminates as well as asking the same model for a probability, but every verdict comes with a readable reason, and its probabilities are calibrated on held-out repositories (ECE 0.023), whereas the raw holistic probabilities are not (ECE 0.17–0.28).
3. **A budget policy with a measured payoff.** Jury-guided early abort increases resolved tasks per agent step by 29% on held-out runs, beats a trace-feature policy, and composes with a tuned step cap (+36%).
4. **A pre-registered study.** Four hypotheses fixed before any judge was run; supported and unsupported results are reported side by side, with all verdicts released.

2. Related work

Benchmarks and agents. SWE-bench [1], SWE-agent [2] and OpenHands [3] define the task; SWE-Gym [4] and R2E-Gym [5] provide training environments and SWE-RL [6] post-trains with RL. Test-based labels are imperfect [7].

Verifiers and judges for code agents. SWE-Gym and R2E-Gym train verifiers over *finished* trajectories; SWE-RM [8] is an execution-free reward model (30B MoE) for test-time scaling and RL; Dockerless [9] verifies *final patches* agentically without environments; Trae Agent [10] uses selection for test-time scaling. All of these judge completed attempts. Early termination of unpromising generations exists for single responses (Speculative Rejection [20]) and process reward models score intermediate reasoning steps [21]; we bring early verdicts to multi-step *agent* trajectories, with judges small enough to share the accelerator with the agent.

LLM-as-a-judge and juries. LLM judges [11] and Agent-as-a-Judge [12] are widely used; Verga et al. [13] replace one large judge with a panel of models, and Zhu et al. [14] study which judges to call and when to stop. Our "jury" is one small model scoring several *task-specific dimensions* with a learned aggregation, inspired by multi-criterion rubric tribunals (BenchClaw; conceptual design only, no code reused). Calibration follows Guo et al. [15].

3. Method

Data. `nebius/SWE-agent-trajectories` [16] (CC-BY-4.0): 80,036 SWE-agent runs on 1,276 Python repositories with a binary "resolved" label from hidden tests. We draw one run per repository and outcome (seed 20261027): 1,765 runs, 27.8% resolved, generated by Llama-3.1 70B (95%), 8B and 405B. Repositories are split 60/40 into development (1,063 runs) and test; the test set is 600 runs stratified by outcome (166 resolved). Patches, evaluation logs and exit status are removed; a check asserts that no repository appears in both splits and that no prefix contains the final `submit`.

Prefixes. A prefix contains the issue and the first k agent actions with their observations. Only runs longer than k steps are judged, so the prefix never reveals the ending: 429 test runs at k = 10 (112 resolved) and 239 at k = 20 (38). Fractional prefixes are reported only as an upper bound (Section 4.1).

Judges. Gemma 4 E2B-it and E4B-it [17] (bf16, vLLM 0.32 [18], temperature 0, fixed seed) receive the same rendered log (messages truncated to fit a 13k-token budget). The *holistic* judge returns `{"p_success": p}`; the *jury* returns six integers 1–5:

dimension	question
localization	has the agent found the code that must change?
reproduction	has it reproduced or understood the bug?
coherence	do the edits and plan match the issue?
test_evidence	do executed tests/scripts support the current fix?
loop	5 = progressing, 1 = repeating commands or failing edits
scope	5 = focused minimal changes, 1 = unfocused

A logistic regression on the six scores (regularization chosen by repository-grouped 5-fold CV on development data) maps them to P(success). A *stack* adds the trace-feature score as a seventh input. As a control for the model family we run Qwen3-4B-Instruct-2507 [19] with byte-identical prompts. Prompts were frozen after a 50-run development pilot (100% valid JSON); 20,596 Gemma verdicts were produced with one invalid output.

No-LLM control. 43 features computed from the prefix only (step counts, command mix, repeated commands, tracebacks, rejected edits, test outcomes, files opened); logistic regression or gradient boosting, chosen on development data.

Early-abort policy. At step k the agent aborts if $1 - P(\text{success}) > \tau$; τ is chosen on development out-of-fold predictions. Efficiency is resolved tasks per agent step, an aborted run costing k steps and resolving nothing; we report the relative gain over never aborting on all 600 test runs (runs shorter than k are never aborted). Controls: the same policy on trace features, step caps chosen on development data, and an issue-only judge that sees no agent step.

Statistics. AUROC, ECE (10 bins) and Brier with 95% CIs from 2,000 repository-cluster bootstrap resamples, paired for differences; two-sided bootstrap p-values; Holm correction over the 8 confirmatory contrasts (H1, H2a, H2b, H4 at k = 10 and 20). Nothing was tuned on the test split.

4. Results

4.1 Early prediction (H1, H2)

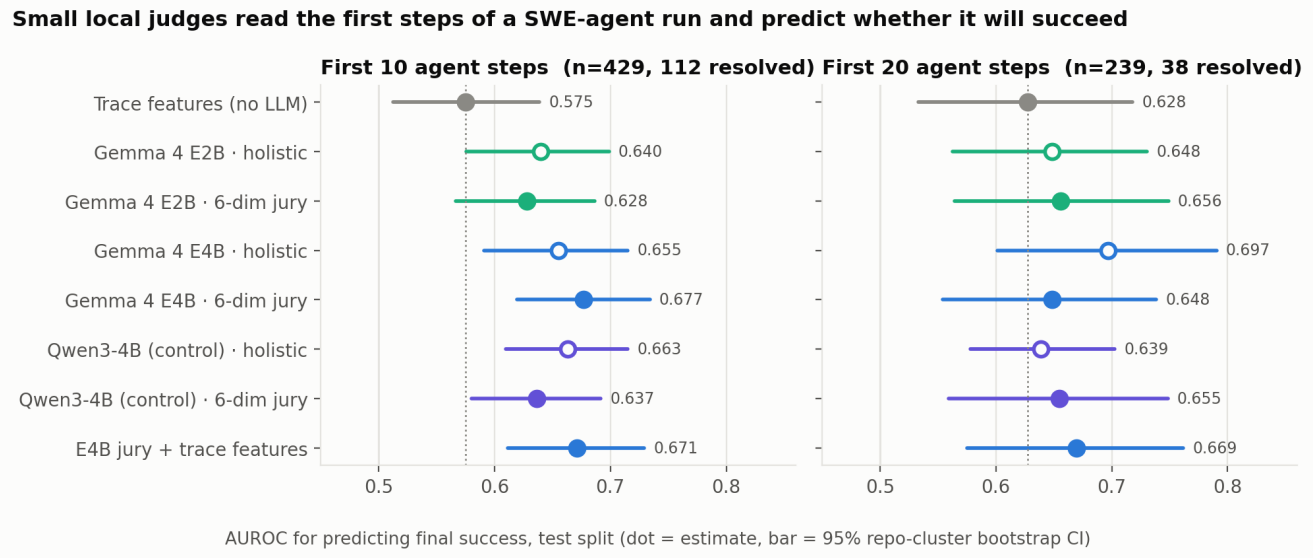


Figure 1: AUROC for predicting final success from the first k agent steps (test split, 95% repository-cluster bootstrap CIs). Hollow markers: holistic judge; filled: six-dimension jury.

AUROC on test [95% CI]	k = 10 (n = 429)	k = 20 (n = 239)
trace features, no LLM	0.575 [0.510, 0.636]	0.628 [0.534, 0.721]
Gemma 4 E2B holistic / jury	0.640 / 0.628	0.648 / 0.656
Gemma 4 E4B holistic	0.655 [0.592, 0.719]	0.697 [0.604, 0.791]
Gemma 4 E4B jury	**0.677 [0.619, 0.734]**	0.648 [0.555, 0.744]
E4B jury + trace features	0.671 [0.612, 0.728]	0.669 [0.573, 0.761]
E4B holistic + trace features	0.652 [0.589, 0.715]	0.712 [0.617, 0.805]
Qwen3-4B holistic / jury (control)	0.663 / 0.637	0.639 / 0.655

(CIs from `results/metrics_preregistered.csv`.) At k = 10 the E4B jury beats the no-LLM control by **+0.102** [+0.034, +0.170] (Holm p = 0.020), and stacking it with trace features beats the control by +0.096 [+0.041, +0.151] (Holm p = 0.006). **H1 as pre-registered (k = 20) is not supported**: with only 38 resolved runs the difference (+0.021) has a CI from −0.09 to +0.13; k = 10 entered the confirmatory family only through the Holm correction added after the pre-registration. The rubric does not discriminate better than asking the same model for a probability (k = 10: +0.022 [−0.021, +0.065]); H2 holds only through its stacking leg, which the holistic stack also passes (+0.077 [+0.011, +0.142]). The rubric's value is interpretability and calibration (Sections 4.2, 4.5), not extra ranking power.

Run or issue? Part of any early signal could come from the issue text alone. An ablation in which E4B sees the same issue but no agent step reaches only AUROC 0.533 [0.482, 0.582] (holistic) and 0.535 (jury); reading the first ten steps adds **+0.142** [+0.063, +0.218] ($p < 0.001$). Among the 52 issues with both a resolved and a failed judged run, the jury ranks the resolved run higher 54% of the time [40, 67], too few pairs to conclude. The early signal comes from what the agent does, not issue difficulty alone.

Why absolute prefixes matter. With fractional prefixes, trace features reach AUROC 0.720 at 66% of the run, because the number of steps in "66% of the run" encodes the total length, which is strongly associated with failure; there the jury adds nothing over trace features (−0.015 [−0.062, +0.030]). A live monitor never knows the final length; absolute prefixes are the honest setting, and the one where the LLM judge helps.

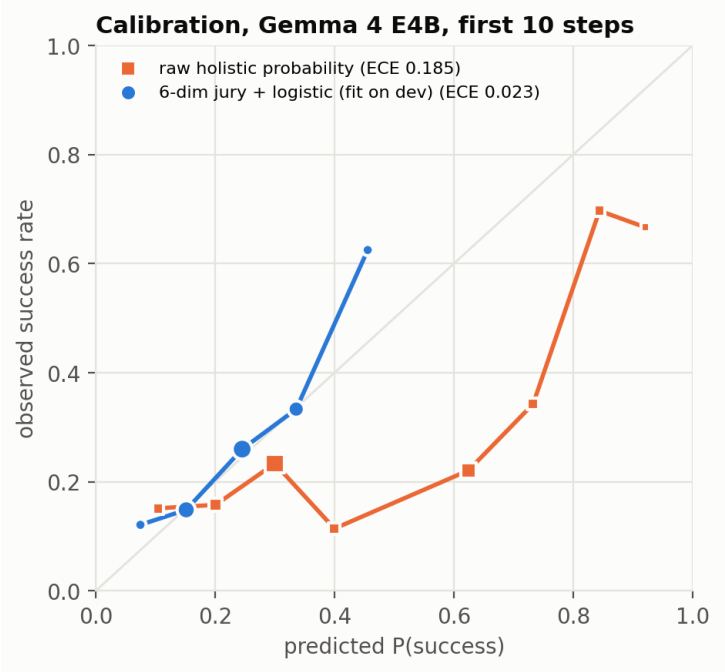


Figure 2: Reliability diagram, Gemma 4 E4B at k = 10: raw holistic probabilities vs. the jury with a logistic layer fitted on development data. Marker area is proportional to bin size.

4.2 Calibration

Raw holistic probabilities from a small model are badly calibrated (E4B ECE 0.185 at k = 10; E2B 0.250): they cluster on round values and overestimate success. The jury's logistic layer, fitted only on development repositories, gives **ECE 0.023** [0.019, 0.070] on the test repositories (Brier 0.178, calibration slope 1.12). That is at the noise floor of this sample size (a perfectly calibrated predictor scores a median ECE of 0.032 here). Re-weighted to the full dataset's 16.7% success rate, ECE rises to 0.072, so the layer should be refitted when the base rate changes. Platt scaling of the holistic score also calibrates (ECE 0.079 at k = 10); the jury adds a per-dimension explanation.

4.3 Model size, cost and model family (H3, control)

E2B keeps 93% of E4B's jury AUROC at k = 10 (ratio CI [0.86, 1.00]) and 101% at k = 20, at 55–59% of the GPU time (batched throughput: 0.35 s vs 0.64 s per judged run on one RTX 3090). Both models read identical prompts, so token cost is the same and the pre-registered "less than half the cost" criterion fails narrowly: **H3 is rejected as stated**. Either model costs a fraction of a GPU-second per judgement.

Model-family control. Qwen3-4B-Instruct-2507 with byte-identical prompts behaves as a peer: its holistic judge (0.663) matches Gemma 4 E4B's (0.655) and also beats trace features (+0.088 [+0.014, +0.161]), so the effect is not Gemma-specific. With the rubric, Gemma 4 E4B is the strongest of the three small judges (0.677 vs 0.637; +0.040 [−0.001, +0.081]), and only its jury makes early abort clearly profitable at step 10 (Qwen3-4B +14% [−2, +31], E2B +17%, E4B +29%).

4.4 Early abort under a fixed budget (H4)

policy (all thresholds chosen on dev)	resolved per step vs never abort [95% CI]
abort every run at step 10	−17% [−35, +3]
trace-feature abort at step 10	+10% [−3, +24]
best step cap on dev (20 steps)	+25% [+12, +38]
Gemma 4 E4B jury abort at step 10	**+29% [+14, +45]**, Holm p ≤ 0.004
E4B jury at step 10 + cap (30 steps)	**+36% [+19, +54]**

policy (all thresholds chosen on dev)	resolved per step vs never abort [95% CI]
Gemma 4 E2B jury abort at step 10	+17% [+0.2, +34]

CIs from `results/contrasts.csv`; combined row from `results/robustness.json`. The jury policy beats the trace-feature policy by +19 points [+7, +33] (unadjusted p = 0.002). At step 10 it aborts 252 of the 429 judged runs: they would have succeeded 18% of the time, against 38% for the runs it keeps. It gives up 45 of 166 eventual successes but saves 6,251 of 14,328 agent steps (44%), the trade a fixed budget rewards. A step cap is a strong baseline in this data, because runs still going after 20 steps rarely succeed: the jury at step 10 matches the best cap (+4 points [−9, +18]) while deciding ten steps earlier, and the two compose (jury + cap minus jury alone: +7 points [+1.5, +13]). At step 20 the jury adds nothing over the cap (+1.7 [−3.7, +7.9]): **its value is in deciding early**.

Robustness. Charging one agent step per judgement: +23% [+8, +38]. Thresholds within ±0.10 of the chosen τ: +18% to +29%. Re-weighting the test runs to the dataset's 16.7% success rate: jury +35%, cap +29%, combined +42%. The simulation measures resolved issues per step under an open stream of issues; with a fixed issue list, aborted issues need a retry policy.

4.5 Does the jury measure what it says?

We re-rated 80 random test prompts (k = 10) blind to outcomes and to Gemma's scores; four Claude Sonnet sub-agents each rated 20 items with the identical rubric (one LLM reference rating per item, not human experts). E4B agrees substantially on **localization (κw 0.65)** and **loop (0.63)**, moderately on reproduction (0.49) and test evidence (0.45), and poorly on scope (0.22) and coherence (0.13); its mean score correlates with the reference at ρ = 0.73 [0.60, 0.83]. Qwen3-4B matches it on localization and reproduction but not on loop (κw 0.16), the dimension that carries most of E4B's signal. The small judges are lenient (coherence 4.6 vs 2.7 on average), which is consistent with the weak dimensions. On these 80 items (easier than average: the jury reaches 0.77 here vs 0.68 on all 429) the reference's mean rating reaches AUROC 0.83 against 0.73 for the jury's mean rating (+0.10 [0.00, +0.19]) and 0.77 for the calibrated jury (+0.06 [−0.02, +0.14]): a stronger judge may help, and the calibrated 4B jury is already close.

5. Discussion

How to use it. In a budgeted harness (e.g. 12 h on 4×L4 with Gemma 4 as the agent), the jury reuses the served weights: one extra prompt at step 10 per attempt, plus a step cap, with τ and the cap tuned on development runs. The dimension scores give a readable reason ("stuck in a loop", "has not found the code") for inspecting runs or filtering SFT/RL data.

What generalizes. The method needs only a log and a rubric: no environment, tests or final patch. It applies to any budgeted agent loop (test-time scaling, best-of-n, truncatable RL rollouts); the prefix-leakage finding applies to any online monitor.

Limitations. (i) All trajectories come from one scaffold (SWE-agent) driven by Llama-3.1 models; transfer to the Gemma 4 competition harness is not measured here. (ii) H4 is a simulation on recorded runs. (iii) The k = 20 analysis is under-powered. (iv) The rubric validation uses an LLM reference. (v) Test-based labels inherit their noise [7]. (vi) The pre-registration was kept in time-stamped project notes, not on a public registry; deviations are listed in the repository.

6. Reproducibility

Code (Apache-2.0), prompts, pre-registration with declared deviations, experiment log (including discarded runs), all verdicts with run ids, and figure scripts: **https://github.com/Agnuxo1/small-juries-early-verdicts**. One RTX 3090 per judge job, vLLM 0.32.0, temperature 0, fixed seeds; job ids and per-call token and time logs are included.

References

[1] Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770.
[2] Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793.
[3] Wang et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741.

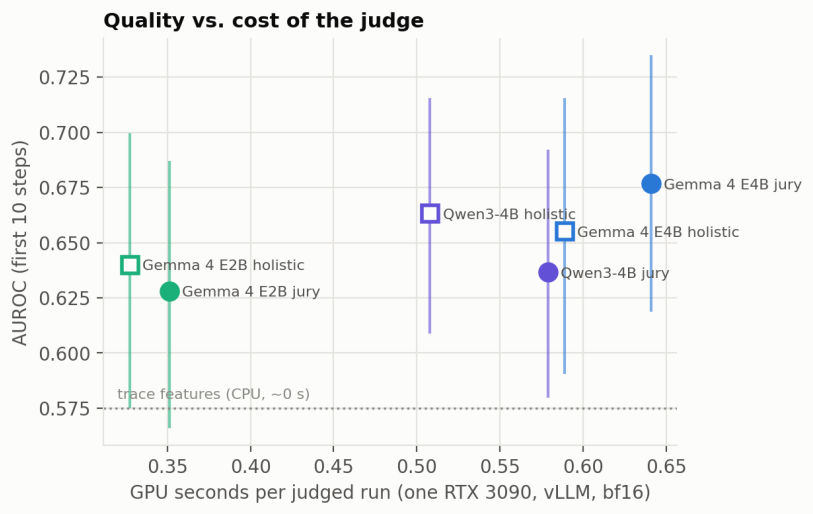


Figure 3: Discrimination vs. GPU time per judgement (k = 10, batched vLLM throughput on one RTX 3090). Hollow: holistic; filled: jury.

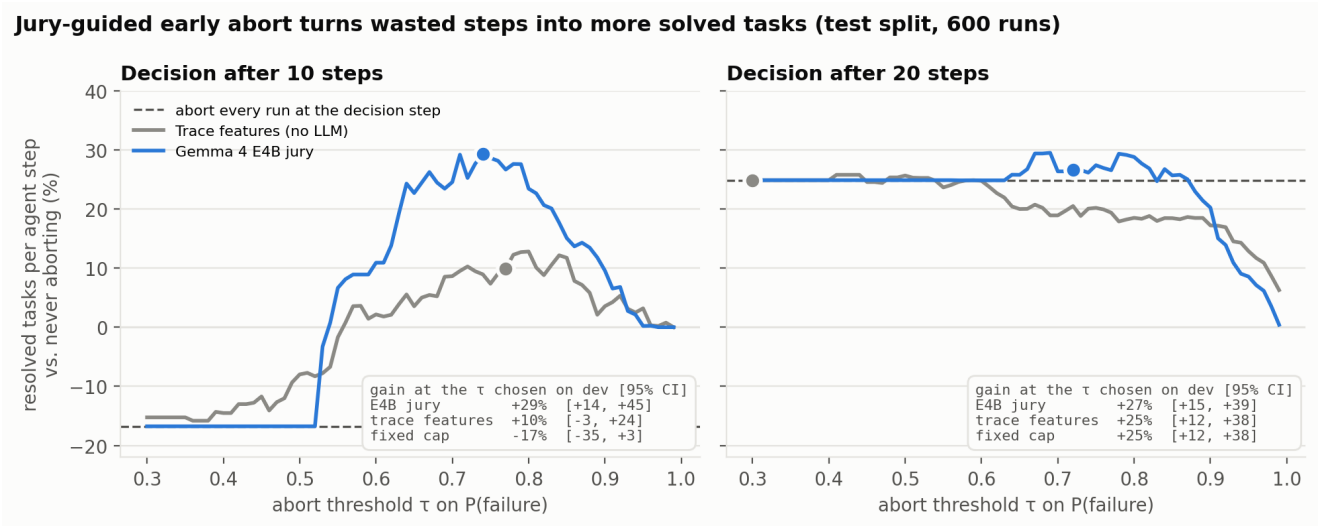


Figure 4: Resolved tasks per agent step relative to never aborting, as a function of the abort threshold (test split). Dots mark the threshold chosen on development data.

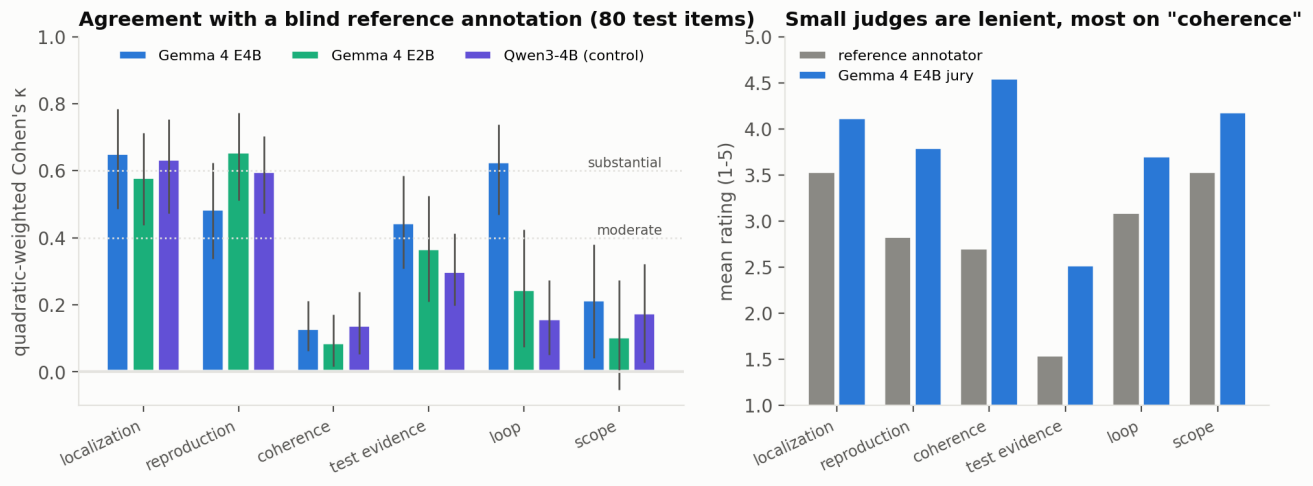


Figure 5: Left: agreement (quadratic-weighted kappa, 95% CI) between the Gemma jury and a blind reference annotation of 80 test prompts. Right: mean rating per dimension.

[4] Pan et al. Training Software Engineering Agents and Verifiers with SWE-Gym. arXiv:2412.21139.
[5] Jain et al. R2E-Gym: Procedural Environments and Hybrid Verifiers for Scaling Open-Weights SWE Agents. arXiv:2504.07164.
[6] Wei et al. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. arXiv:2502.18449.
[7] Wang et al. Are "Solved Issues" in SWE-bench Really Solved Correctly? arXiv:2503.15223.
[8] Shum et al. SWE-RM: Execution-free Feedback for Software Engineering Agents. arXiv:2512.21919.
[9] Zeng et al. Dockerless: Environment-Free Program Verifier for Coding Agents. arXiv:2606.28436.
[10] Trae Research Team. Trae Agent: An LLM-based Agent for Software Engineering with Test-time Scaling. arXiv:2507.23370.
[11] Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685.
[12] Zhuge et al. Agent-as-a-Judge: Evaluate Agents with Agents. arXiv:2410.10934.
[13] Verga et al. Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models. arXiv:2404.18796.
[14] Zhu et al. Stopping and Routing LLM Judge Panels. arXiv:2608.19802.
[15] Guo et al. On Calibration of Modern Neural Networks. arXiv:1706.04599.
[16] Nebius. SWE-agent-trajectories (dataset). huggingface.co/datasets/nebius/SWE-agent-trajectories.
[17] Gemma Team. Gemma 4 Technical Report. arXiv:2607.02770.
[18] Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:2309.06180.
[19] Yang et al. Qwen3 Technical Report. arXiv:2505.09388.
[20] Sun et al. Fast Best-of-N Decoding via Speculative Rejection. arXiv:2410.20290.
[21] Lightman et al. Let's Verify Step by Step. arXiv:2305.20050.